# Platform plan: Cosmos, Blob, Service Bus, and the apps / module / resources layout

Status: **agreed approach**, updated 2026-11-07 (evening). Cosmos for **NoSQL** is a client requirement. §7 shows what is built per phase; §12 lists what is still missing. Settings for Cosmos, Service Bus and the API key are in `core-pipeline/.env` (values never in chat).

**Built so far (all on today's Postgres):**

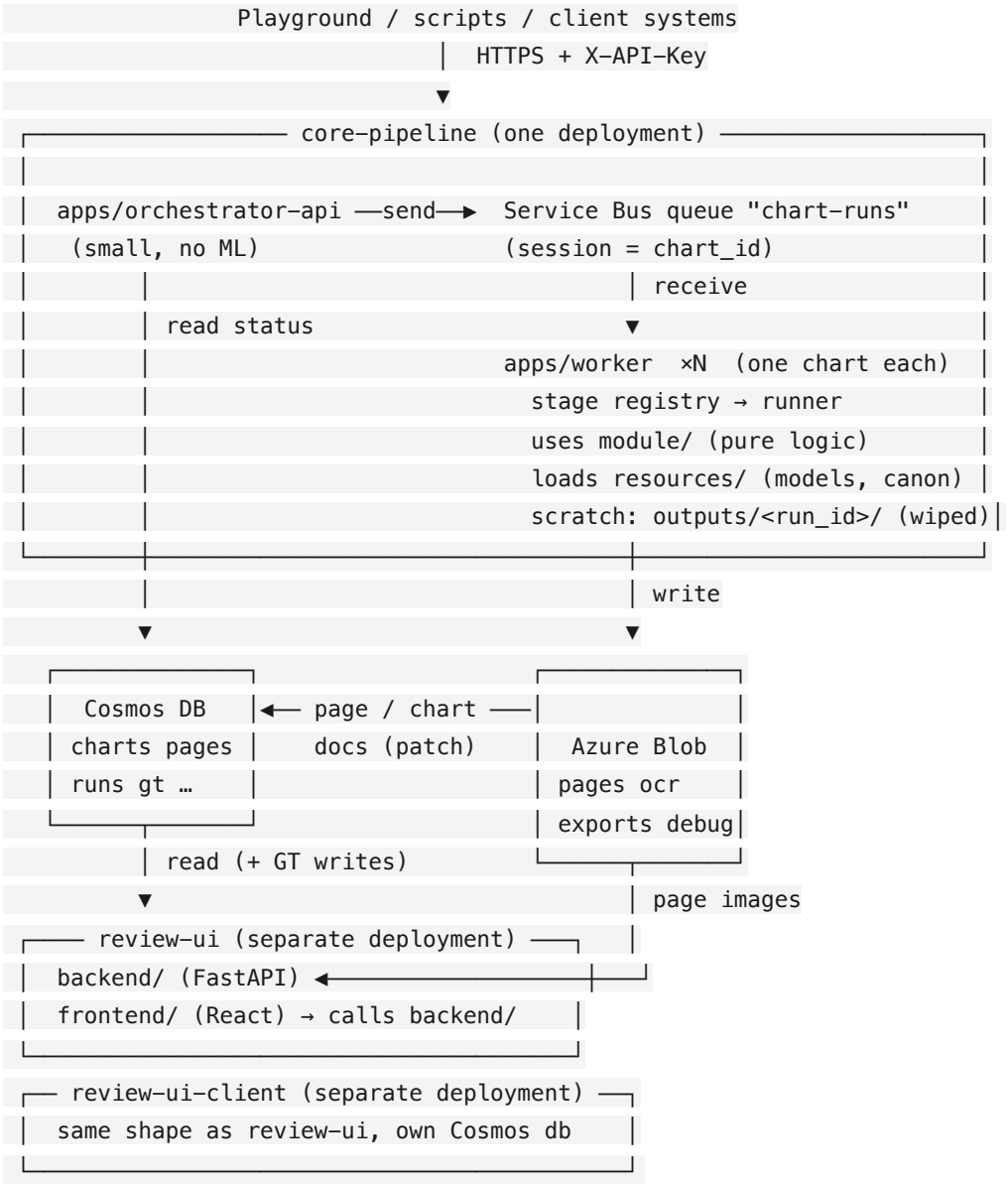
- **Layout (phase 1):** core-pipeline `apps/orchestrator_api/`, `apps/worker/`, `common/`, `modules/`, `resources/canon/`; review-ui `backend/app/{routers,store}/` and a frontend with its own `BACKEND_URL`. About 3,600 lines of dead code removed, and more on 2026-11-07 (old blank/junk training code, the unused sequencing cross-encoder).
- **review-ui reads only the database and Blob.** No Local mode, no File Viewer.
- **`core-pipeline/outputs/` is scratch:** a chart folder not run for 7 days is deleted. Logs are deleted after 7 days.
- **One schema file** (`schema/schema.sql`) plus `schema/migrate.sql` for existing databases.
- **`DB_BACKEND=postgres|cosmos|both`** is in both `.env` files, checked at start and shown in `/health`. Only Postgres is wired: `cosmos` and `both` still run on Postgres.
- **db-utilities** (new, 2026-11-07): manifests and ground truth are loaded by their own Docker job, not by core-pipeline (§3).
- **Monitoring on Postgres:** a `run_usage` row per chart run, daily and hourly views, `python cli.py usage` (§5a).
- **`deploy/`:** the GitHub → Azure DevOps copy leaves out `training/`, `scripts/`, `tests/`.

## 1. Goals

1. Move the database from Postgres to **Azure Cosmos DB** (NoSQL API, provisioned throughput).
2. Keep page images, OCR JSON and exports in **Azure Blob**. Blob is kept forever for now.
3. Manage run requests through **Azure Service Bus** queues.
4. Reorganise core-pipeline into `apps/orchestrator_api`, `apps/worker`, `modules/` and `resources/`. review-ui keeps `frontend/` and `backend/`, each deployable on its own. All Python and TypeScript (no .NET).
5. **core-pipeline, review-ui and review-ui-client are separate infrastructure.** review-ui's frontend and backend can also be deployed apart. Each has its own code, `.env`, Dockerfiles and `infra/`. They share no code at runtime, only the data in Cosmos and Blob.
6. **Models and keyword canons are resources, not code.** They live in `core-pipeline/resources/` and are versioned and replaced without a code release.
7. Write just enough: one source of truth per fact. Debug files are written only in `--debug` mode, **to both** local disk (`core-pipeline/outputs/debug/`) and Blob (`debug/`). A debug run still writes all its results to Cosmos as usual.
8. **No more output to `review-ui/data`.** `core-pipeline/outputs/` is temporary scratch. **review-ui reads only Cosmos and Blob; Local mode is removed.**
9. Step-by-step APIs (OCR, page type, member verification, DOS) that run only the stages they still need.
10. **Monitoring:** every run records time taken, Doc Intel pages and cost, and LLM tokens and cost (once an LLM is used), in one place with daily and hourly totals.
11. **Reference data is loaded by its own job.** Manifests and ground truth go into the database through **db-utilities**, a separate Docker job, never through core-pipeline or review-ui.
12. **Models come from Blob,** not from a manual copy (phase 8), so every host and every pod gets identical weights.

13. **AKS later.** Every part runs as a container today, so the move to Azure Kubernetes Service is a deployment change, not a code change (§8a).

## 2. Overall approach




A run, step by step:

1. A caller posts to the orchestrator: a full run, or a target such as `/member-verify`. The orchestrator checks the API key, writes a `runs` doc, puts one message on `chart-runs` and returns `202 + run_id`.
2. A worker takes the message. Service Bus sessions guarantee that one chart is processed by only one worker at a time.
3. The worker works out which stages the target needs and skips those already done with the same `version` and `input_hash`. It runs the rest.
4. Each stage reads its inputs from Cosmos and Blob, runs `module/` code with `resources/` loaded, and writes its result as one patch on the page doc. Large JSON goes to Blob.
5. The worker marks the run done and completes the message. Its scratch folder `outputs/<run_id>/` is deleted. With `debug`, the explanation files are also written to `outputs/debug/<run_id>/` on that machine's disk and to Blob `debug/<run_id>/`.
6. review-ui shows the chart from Cosmos, with images from Blob. Ground truth saves go to Cosmos.

## 3. Target structure: separate deployments

```
advantmed-imaging-pipeline/                (one GitHub repo; three deployables)
├── core-pipeline/                         ── DEPLOYMENT 1 ──
│   ├── apps/
│   │   ├── orchestrator_api/              FastAPI: auth, enqueue, status, exports
│   │   │   ├── app/{main.py,routers/,services/}
│   │   │   ├── requirements.txt
│   │   │   └── Dockerfile
│   │   └── worker/                        Service Bus consumer, runs the chain
│   │       ├── worker/{main.py,pipeline/,stages/}
│   │       ├── requirements.txt          (torch, docling, gliner … only here)
│   │       └── Dockerfile
│   ├── modules/                           the logic (was stages/lib; moved 2026-11-07)
│   │   ├── image_preprocess/ ocr/ blank_junk/ page_classify/ extraction/
│   │   └── member/ dos/ continuity/ encounter/ sequencing/ imaging_final/
│   ├── common/                            config, logging, db/ (today); cosmos/, blob/,
servicebus/,
│   │                                      artifacts.py, contracts/ (doc models)
│   ├── resources/                         local cache of Blob resources/ (git-ignored)
│   │   ├── manifest.json                  the live versions, as last read from Blob
│   │   ├── models/                        weights, loaded at worker start
│   │   │   ├── hw/ rapidocr/ docling/ semantic-model/ ner/
│   │   │   └── blank-junk/ page-family/ encounter/ kv-extraction/
│   │   └── canon/                         keyword JSONs, checked before each run
│   ├── outputs/                           scratch per run (wiped) + debug/ (debug mode only)
│   ├── logs/                              7-day retention
│   ├── infra/                             Cosmos db+containers, SB queue (Bicep)
│   ├── tests/
│   ├── docker-compose.yml                 orchestrator-api + worker ×N
│   └── .env / .env.example
│
├── review-ui/                             ── DEPLOYMENTS 2a + 2b ──
│   ├── backend/                           2a: FastAPI, deployable on its own
│   │   ├── app/
│   │   │   ├── main.py
│   │   │   ├── routers/                   charts, pages, ground_truth, accuracy, users …
│   │   │   ├── services/                  accuracy, excel_compare, labels (no queries)
│   │   │   ├── store/                     postgres/ and cosmos/ (DB_BACKEND), blob.py
│   │   │   └── contracts/                 copy of core-pipeline's doc schemas
│   │   ├── tests/  requirements.txt  Dockerfile  .env / .env.example
│   ├── frontend/                          2b: React, static build, deployable on its own
│   │   ├── src/{pages,components,api,lib}/
│   │   ├── package.json  Dockerfile (nginx)  .env.example (API URL)
│   ├── infra/                             one file per part (backend, frontend)
│   └── docker-compose.yml                 both together, for one-host installs
│
├── review-ui-client/                      ── DEPLOYMENT 3 ── (ui-temp repo, synced)
│
├── db-utilities/                          ── DEPLOYMENT 4 ── (a job, not a server; built)
│   ├── dbutil/                            manifest, metadata-extra, ground-truth, sequence
```

```
│   ├── Dockerfile  docker-compose.yml   `docker compose run --rm db-utilities <job> …`
│   ├── input/                           files to load (read-only at /input), or a Blob prefix
│   └── .env / .env.example              DATABASE_URL (later Cosmos) + AZURE_STORAGE_*
│
├── k8s/                                 (phase 9) AKS manifests or Helm chart, one per deployment
├── schema/                            Postgres: schema.sql, migrate.sql, migrations/
├── deploy/                            GitHub → DevOps copy, client .gitignore
├── training/  scripts/  tests/        GitHub only, never on the VM
├── utilities/   docs/
└── README.md
```

### Rules

- **`modules/` never touches I/O.** It takes text, boxes and settings in, and returns results.
- **Only `worker/stages/` adapters read and write storage,** always through `imaging_common`.
- **`orchestrator_api` never imports `modules`** and has no ML dependencies, so it stays small and starts fast.
- **Code never hard-codes a model path or a canon file.** It asks `resources` for `models/page-leaf` or `canon/continuity_canon` and gets the version from `manifest.json`. The version is stamped on every stage result.
- **The deployments share data, not code.** core-pipeline owns the Cosmos document shapes (`imaging_common/contracts`, pydantic). Their JSON Schemas are copied into `review-ui/backend/app/contracts/`, and a test fails when the two differ. `utilities/sync_client_ui.sh` keeps doing the copy for review-ui-client.
- **Nothing writes into another deployment's folder.** The `review-ui/data/folders` hand-off has ended (built).
- **review-ui's frontend talks only to its backend**, over HTTP at the URL set at build time. It never reads the database or Blob itself, so the two parts can run on different hosts, or the frontend from static hosting.

### Resources: models and canon (from Blob)

Not built yet (phase 8). On the VM today the weights are copied by hand into `core-pipeline/models/`.

Both live in Blob as versioned folders. A small `manifest.json` says which version is live.

```
resources/                                  (Blob container)
├── manifest.json                           item -> live version + file fingerprints
├── models/<name>/<version>/…               e.g. models/page-leaf/v3/
└── canon/<version>/*.json                  e.g. canon/2026-11-07a/
```

| | Models | Canon |
|---|---|---|
| What | trained weights (BERT encoders, GLiNER, RapidOCR, Docling, MiniLM, handwriting) | keyword JSONs: section headers, continuity, duplicates, leaf keywords … |
| Source of truth | Blob only (too big for git) | git, so tests run on it and every edit has a history; `utilities/publish_resources.py` uploads a version to Blob |
| When a worker reads it | **once, when the worker starts** | **before each chart run** (one small manifest read; downloads only on a change) |
| Local copy | `core-pipeline/resources/` on the host, shared by every worker on it | same |
| Changing one | upload a new version folder and change `manifest.json`; takes effect on the next restart, never mid-batch | publish or upload, change `manifest.json`; live for the next chart, no redeploy |
| Rolling back | point `manifest.json` at the old version | same |

- **A half-finished upload can't be used:** the worker checks each file against the fingerprint in the manifest and refuses a mismatch.
- **Every result records its versions.** Each stage stamps the model and canon versions it ran with into `input_hash`, so a canon change re-runs exactly the stages it affects.
- **If Blob can't be reached,** the worker keeps what's already on disk and records that in `/health` and in the run record.
- **What it removes:** copying model weights to the VM by hand (the first worker start downloads about 3 GB, once), and the zip → DevOps → VM round trip for a canon edit.
- **Weights are never built into a Docker image:** an image would be many GB, every model change would need a new image, and every new AKS node would pull it again.
- **On AKS** the same fetch runs as an **init container** that fills a shared volume before the workers start (or the Blob CSI driver mounts `resources/models` read-only), §8a.

### Where today's code goes

| Today | Moves to |
|---|---|
| `core-pipeline/modules/*` (logic) | `core-pipeline/modules/*` |
| `core-pipeline/resources/canon/` | git copy stays in the repo; published to Blob `resources/canon/` |
| `core-pipeline/models/` | Blob `resources/models/`, cached in `core-pipeline/resources/models/` |
| `core-pipeline/stages/*` runners, `core-pipeline/apps/worker/runner.py` | `core-pipeline/apps/worker/worker/{stages,pipeline}/` |
| `core-pipeline/apps/orchestrator_api/main.py` | `core-pipeline/apps/orchestrator-api/app/` (split into routers) |
| `core-pipeline/db/*`, `blob_store.py`, `paths.py` | `core-pipeline/common/imaging_common/{cosmos,blob}/` |
| `review-ui/data/folders` (pipeline output) | **done:** scratch in `core-pipeline/outputs/charts`; results from the database + Blob |
| `review-ui/backend/` | stays; split into `routers/`, `services/`, `store/` |
| `review-ui/frontend/` | stays; own Dockerfile and API URL, deployable alone |
| review-ui Local mode (`DATA_MODE=local`, disk CSV/JSON readers) | **done:** removed; the disk readers still inside `store/local` go with the `store/` split |
| `review-ui/.../adapters/postgres/repository.py` | `review-ui/backend/app/store/postgres/`, beside `store/cosmos/` |
| `schema/*.sql` | `imaging_common/contracts` + `core-pipeline/infra/` |

## 4. Data: Postgres → Cosmos, disk → Blob, jobs → Service Bus

### 4.1 Cosmos (NoSQL API, provisioned throughput)

Partition key is `/chart_id` everywhere except users.

| Container | Document | Replaces |
|---|---|---|
| `charts` | One doc per chart: manifest info, status, per-stage state, summary, member verdict, tags | `chart_list`, `chart_status`, member summary, run stats |
| `pages` | One doc per page, with one sub-object per stage (`quality`, `ocr` refs and text, `blank_junk`, `classification`, `kv`, `member`, `dos`, `continuity`, `sequence`, `final`) | ~20 per-page result tables, plus the staging JSONs |
| `runs` | One doc per run: request, stages run, timings, errors | `pipeline_jobs`, `page_stage_status`, run logs |
| `ground_truth` | One doc per page, plus a chart-level doc | the ground-truth tables |
| `users` (pk `/id`), `manifest` (pk `/record_id`), `metrics` | — | users, manifest, accuracy snapshots |

- **Stage writes** use a Cosmos patch on the stage's own sub-object, so two stages never overwrite each other.
- **Stage completion** is recorded as `stages.<name> = {done_at, version, input_hash}`. A redelivered message is then a no-op. `skip_ocr` and the step-by-step APIs read the same record.

- **Size limit:** a document can be at most 2 MB. Word boxes and the full Docling / Azure JSON go to Blob; the page doc keeps the text and the blob path.
- **Throughput:** provisioned, at the database level and shared by the containers, with autoscale. Start at 4,000 RU/s maximum and tune from the metrics. The `pages` container gets its own dedicated throughput only if batch runs throttle it.
- **History is migrated** (§7, phase 7). Every Postgres chart, page result, run, ground truth and user is copied into Cosmos by a one-off utility, and counts are compared before cutover.

### 4.1a Who writes what

Workers write to the database directly. Nothing routes their results through the orchestrator.

| Writer | Cosmos | Blob |
|---|---|---|
| **Worker** | page docs (one patch per stage), chart status and summary, its `runs` doc (stage in progress, timings, errors) | pages, OCR JSON, exports, debug files |
| **Orchestrator API** | creates the `runs` doc; reads status | — (reads exports for downloads) |
| **review-ui** | ground truth, reviews, users, chart tags | — (reads page images) |
| **db-utilities** (jobs) | `manifest` (members per record), `ground_truth` (bulk loads) | — (reads files to load) |
| **Migration utility** (once) | everything copied from Postgres | — |

Each writer has its own key: workers and the orchestrator get read-write access, review-ui gets read-write only where it saves, and reports read with a read-only key.

### 4.2 Blob (everything kept for now)

| Container | Holds |
|---|---|
| `inbox` | the existing raw input (unchanged) |
| `pages` | `<chart>/<page>.png`, rotated / corrected pages: what review-ui shows |
| `ocr` | `<chart>/<page>.{prelim,final1,final2}.json`: what `skip_ocr` reuses and the overlays come from |
| `exports` | final CSV / JSON deliverables |
| `debug` | `<run_id>/…`, written only with `debug: true` (a copy of what goes to local disk) |
| `resources` | model weights by name and version (optional, §3) |

There are no lifecycle rules for now. If storage cost becomes an issue, one rule on `debug` is the only change needed.

### 4.3 Service Bus

- **Queue `chart-runs`, sessions on, with the session id set to `chart_id`.** Two runs of one chart are processed in order and never at the same time. Different charts run in parallel. Sessions need the **Standard** tier or higher.
- **The message** is `{run_id, chart_id, target, only?, skip_ocr, debug, input}`.
- **Long charts:** the worker renews the message lock while it works, because one lock lasts at most 5 minutes. If a worker dies, the message comes back and finished stages are skipped. After 5 failed deliveries the message moves to the dead-letter queue, and the `runs` doc says so.
- **Retired:** the Postgres job table, the lease sweeper and FastAPI background tasks.

### 4.4 The DB toggle (short term)

Until Postgres is retired, `.env` picks the store (built: the setting is checked at start and shown in `/health`; only `postgres` is wired so far):

```
DB_BACKEND=postgres     # today's schema/schema.sql database (default)
DB_BACKEND=both         # write Postgres AND Cosmos, read Postgres: the side-by-side week
DB_BACKEND=cosmos       # the containers in §4.1
```

- core-pipeline, review-ui and db-utilities each read their own `DB_BACKEND`. They must agree, or review-ui shows charts the pipeline is no longer writing.
- Stages, routes and jobs call one interface (`store.charts`, `store.pages`, `store.runs`, `store.ground_truth`, `store.manifest`, `store.users`). Only the implementations know SQL or Cosmos.
- `both` makes every write go to both stores, so the week before cutover compares them on real runs. Reads stay on Postgres until the switch to `cosmos`.
- Switching moves no data. The history migration (phase 7) copies it, and a mismatch check runs before the default changes to `cosmos`.

### 4.4a What Cosmos for NoSQL changes in the code

Cosmos for NoSQL has no joins across containers and no SQL views, so every query that joins tables today is rewritten around the documents in §4.1:

| **Today (Postgres)** | **On Cosmos** |
|---|---|
| core-pipeline `common/db/` (about 2,000 lines of SQL helpers) | `store/cosmos/`: read and patch the chart and page documents |
| review-ui `store/postgres/repository.py` and the services with SQL (accuracy, value search, Manual Review, review log) | `store/cosmos/`: chart / page reads by partition; cross-chart pages read **precomputed** docs in `metrics` |
| Report views (`consolidated_chart_results`, `v_run_usage_*`) | documents written when a run ends, rebuilt by a nightly job |
| `page_stage_status` rows | `stages.<name>` on the page doc (§4.1) |
| db-utilities upserts | upsert into `manifest` / `ground_truth` |

This is the largest single piece of work in the plan (phases 2, 6 and 7).

### 4.5 Auth

The orchestrator API takes an **API key** in the `X-API-Key` header, from `ORCHESTRATOR_API_KEY`. `/health` stays open. review-ui keeps its own sign-in, and its Playground sends the key when it starts runs.

## 5. Write just enough, and `--debug`

| | Written | Where |
|---|---|---|
| **Always** | Page and chart docs | Cosmos |
| **Always** | Page images, OCR JSON, final export | Blob (`pages`, `ocr`, `exports`) |
| **During a run only** | Working files | `core-pipeline/outputs/<run_id>/`, deleted when the run ends |
| **Only with `debug: true`** | `staging/*.json` explanations, per-stage CSVs, the scratch folder | **Both**: local disk `core-pipeline/outputs/debug/<run_id>/` on the worker (deleted after 7 days, like the logs) and Blob `debug/<run_id>/` (kept) |
| **Logs** | `core-pipeline/logs/` | deleted after 7 days (built) |

- **CSVs on demand:** `GET /v1/charts/{id}/export?format=csv&stage=…` builds the CSV from Cosmos when asked.
- **One writer:** `artifacts.write(kind, name, data)` is the only file writer. A test fails if a stage writes a file directly.
- **Data that stages pass to each other moves into the page doc.** Today `staging/extraction.json` and `staging/rotation.json` carry it; those become Cosmos fields.

## 5a. Monitoring: time, Doc Intel cost, LLM tokens

**Today:** `core-pipeline/logs/run_stats.jsonl` gets one line per chart run, with time taken, pages and the number of pages sent to Doc Intel. `run_stats_charts.csv` and `run_stats_hourly.csv` are rebuilt from it, and `python cli.py stats` prints the hourly table. It lives in the log folder, so it's lost when a VM is rebuilt, and it records no cost or tokens.

**Target:** every billable or timed thing is recorded where it happens and kept in the database, never only in a log file.

| What | Recorded by | Fields |
|---|---|---|
| **Time** | the worker, per stage and per run | started, ended, seconds; pages; seconds per page; queue wait (enqueued → started) |
| **Doc Intel** | the Final OCR 2 stage, per page call | pages sent, pages billed, model (`prebuilt-layout` …), features, retries, throttles (429s), cost |
| **LLM** (when one is used) | one wrapper around every LLM call | provider, deployment, prompt tokens, completion tokens, cached tokens, latency, cost, which stage and page |
| **Errors** | the worker | failed stage, retry count, dead-lettered runs |

**Where it is written:**

- **Per run:** a `usage` block on the `runs` document (Cosmos), written as the run goes, so a crash still leaves what was spent.
- **Totals:** a `metrics` container holding one document per day and per hour, with charts, pages, seconds, Doc Intel pages and cost, and LLM tokens and cost, split by trigger, stage and model. It's updated by the worker when a run ends and rebuilt by a nightly job, so a missed update fixes itself.
- **While Postgres is still in use (built):** the same rows go to the `run_usage` table (Part 1 of `schema.sql`), with `v_run_usage_daily` and `v_run_usage_hourly`. Prices: `.env` section 5.

- **Costs come from a price table, not code:** `PRICE_DOCINTEL_PER_1000_PAGES` and `PRICE_LLM_<deployment>_IN` / `_OUT` per 1,000 tokens in `.env`. The price used is stored with each row, so a later price change never rewrites history.

**Reading it:**

- `GET /v1/metrics?from=&to=&group=day|hour|trigger|stage` on the orchestrator API.
- `python cli.py stats` (the same numbers, in the terminal).
- A **Usage** page in review-ui for admins: time per chart, Doc Intel pages and cost per day, and tokens per stage.
- **Alerts** (optional): a daily Doc Intel spend above a set amount, or a run slower than a set time, sends a message through the `run-events` topic.

## 6. Step-by-step APIs

One stage registry with dependencies, for example `member_verify requires [ocr_final2, kv_extract, blank_junk_2]`. Each endpoint names a target. The worker runs only what is missing or stale for that target, then stops.

| Endpoint | Runs up to | Returns |
|---|---|---|
| `POST /v1/charts` (full) | stage 15 | `run_id` |
| `POST /v1/charts/{id}/ocr` | rotation → prelim → final1 → final2 → section headers | per-page text, engine, quality |
| `POST /v1/charts/{id}/page-type` | OCR + blank/junk 1 and 2 + kv + page classification + subtype | per-page type / subtype / blank-junk |
| `POST /v1/charts/{id}/member-verify` | OCR + blank/junk + kv + member | member verdict, wrong-member pages |
| `POST /v1/charts/{id}/dos` | … + DOS | per-page and per-document dates |
| `GET /v1/runs/{run_id}` | — | status, the stage in progress, errors, the result when done |

**How a caller gets the answer.** This was decision 3, and it is now set to the default below. A 400-page chart takes minutes, longer than an HTTP request should stay open. So every endpoint replies at once with `202` and a `run_id`, and the caller asks `GET /v1/runs/{run_id}` until it says done; that reply carries the result. For a small chart, or a target already done, add `?wait=60`: the call waits up to 60 seconds and returns the result directly if it finishes in time. Tell me if a caller needs something else.

- `only`, `skip_ocr` and `debug` work as today.
- **Re-running:** a stage counts as done only if its `version` and `input_hash` match. The hash includes the model and canon versions, so a canon edit re-runs exactly the stages it affects.

## 7. Phases

| # | Phase | What | Done when | Status (2026-11-07) |
|---|---|---|---|---|
| 0 | Access | Cosmos, Service Bus and Blob details in `core-pipeline/.env`; `infra/` (Bicep) creates the database, containers and queue | the code connects to all three from the Mac and from the VM | **Partly:** settings in `.env`, queue `chart-runs` created, Cosmos reachable. No `infra/`, and no code connects to Cosmos or Service Bus yet |
| 1 | Restructure, no logic change | `apps/`, `modules/`, `common/`, `resources/`; review-ui `backend/` + `frontend/` each with its own Dockerfile | same outputs on the test charts; tests pass | **Done** except: the API is still one `main.py` (not split into routers), and the API and worker are one image, one process |
| 2 | Storage layer + DB toggle | one data-access interface, Postgres and Cosmos implementations; `DB_BACKEND=postgres\|cosmos\|both` | both pass the same tests; one chart end to end on each | **Toggle only:** the setting is checked and shown; no interface, no Cosmos implementation |
| 3 | Outputs and debug | every file written inventoried; stage-to-stage data into page docs; `artifacts.write`; scratch wiped per run; debug to disk and Blob | a non-debug run writes only to the DB and Blob | **Partly:** scratch with a 7-day purge; debug goes to local disk only; `staging/*.json` still carries data between stages |
| 4 | Service Bus + API key | orchestrator enqueues, workers consume (sessions, lock renewal, dead-letter); `X-API-Key` | kill a worker mid-chart and the chart finishes on another | **Not started** |
| 5 | Step-by-step APIs | stage graph, version / input-hash check, `/v1/...` endpoints | `/member-verify` runs only what it needs; a second call is instant | **Not started** |
| 6 | review-ui on Cosmos | `store/cosmos/`, precomputed accuracy docs; sync review-ui-client | the test charts look the same as today | **Not started** (DB-only and Blob images: done) |
| 7 | History migration and cutover | `utilities/migrate_postgres_to_cosmos.py`; `DB_BACKEND=both` for a week; db-utilities writes Cosmos; Cosmos becomes the default | counts match; Postgres read-only, then retired | **Not started** |
| 7a | Monitoring | `run_usage` now; `runs.usage` + `metrics` on Cosmos; LLM wrapper; `/v1/metrics`; review-ui Usage page | every run shows time, Doc Intel pages and cost | **Partly:** `run_usage` and `cli.py usage` built; no LLM wrapper, `/v1/metrics` or Usage page |
| 8 | Resources from Blob | `manifest.json`, `publish_resources.py`, models fetched at worker start, canon checked before each run, | a new model reaches every | **Not started** (weights still copied by hand) |

| # | Phase | What | Done when | Status (2026-11-07) |
|---|---|---|---|---|
| | | fingerprints | host with no manual copy | |
| 9 | AKS | images in ACR; one Deployment per service; workers scaled by queue length; db-utilities as Kubernetes Jobs; secrets from Key Vault (§8a) | the VM's `docker compose` setup runs unchanged on AKS | **Not started** (needs phases 4 and 8 first) |
| — | db-utilities | manifests and ground truth loaded by their own Docker job | core-pipeline has no load code | **Done** on Postgres (2026-11-07); Cosmos writer in phase 7 |

Phase 1 goes first because every later change then lands in its final place, and its outputs can be compared one for one. Phases 4 and 8 come before 9: AKS needs workers that pull from a queue and models that come from Blob.

## 8. Running and deploying

Development happens on the Mac. On the client side the code arrives as a **zip downloaded from GitHub on the Windows laptop** (no git there), goes to **Azure DevOps**, and is deployed on the **Ubuntu VM (16 CPU, 64 GB RAM)**. Every round trip is slow, so each phase ships as one zip with every change for that phase.

**Windows laptop (PowerShell): refresh the code.** `deploy/prepare_for_devops.ps1` copies the GitHub zip into the DevOps folder without `training/`, `scripts/` and `tests/`, keeps `.env` files and weights, and installs `deploy/devops.gitignore`. Full steps are in `deploy/README.md`.

```
powershell -ExecutionPolicy Bypass -File "$env:USERPROFILE\Downloads\model-repo-main\deploy\prepare_for_devops.ps1" `
    -Source "$env:USERPROFILE\Downloads\model-repo-main" -Target "C:\work\advantmed-devops"
# then push C:\work\advantmed-devops to Azure DevOps as usual
```

**Ubuntu VM (bash): deploy core-pipeline** (each deployment is started from its own folder)

```
cd ~/advantmed-imaging-pipeline/core-pipeline
# after pulling the new code from Azure DevOps
docker compose build
docker compose up -d --scale worker=3     # 3 workers × 1 chart each
curl -fsS localhost:8001/health
docker compose logs -f worker
```

**Ubuntu VM (bash): deploy review-ui** (both parts on one host; each can also run alone)

```
cd ~/advantmed-imaging-pipeline/review-ui
docker compose up -d --build                  # backend + frontend
docker compose up -d --build backend          # backend only
docker compose up -d --build frontend         # frontend only (API URL set in its .env)
```

**Sizing the 16 CPU / 64 GB VM.** One worker holds Docling, GLiNER and the BERT models at about 10–14 GB, and uses 4 to 5 CPUs well. Three workers plus the orchestrator API and review-ui fit with headroom. Start at `--scale worker=3` and go to 4 only if memory stays under 48 GB during a batch. Because jobs come from Service Bus, adding a worker on another machine later needs no code change.

## 8a. AKS (later)

Every part already runs as its own container, so moving from the VM to Azure Kubernetes Service changes how they are started, not the code.

| Part | On AKS |
|---|---|
| orchestrator API | a Deployment (2 replicas) behind an internal Service / Ingress; small, no models |
| worker | a Deployment, **scaled by KEDA on the length of the `chart-runs` queue** (0 → N pods); one chart per pod; a node pool with enough memory (about 14 GB per worker) |
| models | an **init container** copies `resources/models/<version>` from Blob into a volume the worker mounts read-only; or the **Blob CSI driver** mounts it directly. Never inside the image |
| review-ui backend / frontend | two Deployments + an Ingress (TLS); the frontend's `BACKEND_URL` points at the backend Service |
| db-utilities | a Kubernetes **Job** per load (`kubectl create job … -- manifest --blob-prefix …`), reading files from Blob instead of `./input` |
| secrets | Azure **Key Vault** through the Secrets Store CSI driver, with **workload identity** for Blob, Cosmos and Service Bus (no keys in files) |
| images | Azure Container Registry, built by the Azure DevOps pipeline |
| logs | stdout to Azure Monitor / Container Insights instead of `core-pipeline/logs/` |

What has to exist first: the queue and separate worker (phase 4), models from Blob (phase 8), and settings read from the environment only (already true).

## 9. Decisions (2026-11-07)

| # | Question | Decision |
|---|---|---|
| 1 | Cosmos throughput | **Provisioned** (database level, autoscale) |
| 2 | History | **Migrate everything** from Postgres |
| 3 | Step-by-step APIs: sync or async | **Default chosen:** async with an optional `?wait=` (§6) |
| 4 | Blob retention | **Keep everything for now**, debug included. Debug files go to both local disk and Blob |
| 5 | Orchestrator API auth | **API key** (`X-API-Key`) |
| 6 | Models and canon | **Resources inside core-pipeline** (`resources/`), outside the code |
| 7 | Deployments | **core-pipeline, review-ui and review-ui-client are separate infra** |
| 8 | Pipeline output | `core-pipeline/outputs/` is temporary; **review-ui is DB only, no Local mode** |
| 9 | Short term | **A `DB_BACKEND=postgres\|cosmos` toggle** in core-pipeline and review-ui until Postgres is retired |
| 10 | Client copy | `training/`, `scripts/`, `tests/` stay on GitHub only; DevOps and the VM get the runtime (`deploy/`) |
| 11 | Logic folder name | `core-pipeline/modules/` (not `model/`), imported as `modules.<concern>` (built 2026-11-07) |
| 12 | review-ui layout | `frontend/` and `backend/` stay, each deployable on its own; the frontend talks only to the backend |
| 13 | Models and canon delivery | Blob `resources/` with `manifest.json`: models loaded at worker start, canon checked before each run; canon also kept in git |
| 14 | Debug files | Written to both local disk and Blob `debug/` |
| 15 | Monitoring | Time, Doc Intel pages and cost, LLM tokens and cost: recorded per run and kept as daily and hourly totals in the database (§5a) |
| 16 | Database | **Cosmos for NoSQL**: a client requirement (evening of 2026-11-07) |
| 17 | DB toggle values | `postgres`, `cosmos`, or `both` (write both, read Postgres) |
| 18 | Manifests and ground truth | Loaded only by **db-utilities**, a separate Docker job; not part of core-pipeline or review-ui |
| 19 | Models | Fetched from Blob at start (phase 8); never built into images |
| 20 | Hosting | The Ubuntu VM now; **AKS later** (§8a) |

## 10. What I need from you

Already in `core-pipeline/.env`: the Cosmos endpoint, key and database; the Service Bus connection string and queue; the API key; the Blob account. Still open:

**Cosmos DB**

- Is 4,000 RU/s autoscale (maximum) acceptable to start?
- A read-only key for review-ui, or the same key?
- Public access from the VM and the Mac, or a private endpoint only?
- review-ui-client: its own database in the same account, or another account?
- Permission to create the containers in §4.1.

**Blob**

- A `resources` container for models (phase 8), or a folder in `imaging-pipeline`?
- On the VM: a managed identity (preferred) or a key?

**AKS (when you get there)**

- The cluster, its container registry (ACR) and whether Key Vault is the rule for secrets.

## 11. Risks

| Risk | Answer |
|---|---|
| Cosmos cost on review-ui's cross-chart accuracy pages | Precomputed metric docs in `metrics` |
| The 2 MB document limit | OCR JSON and word boxes stay in Blob |
| Duplicate deliveries | Stage completion is recorded per stage, so a redelivered message skips finished work |
| Worker memory with GLiNER, Docling and BERT together | One chart per worker; at most 3–4 workers on the 64 GB VM |
| Slow client round trip (zip → DevOps → VM) | One zip per phase; models from Blob (phase 8) remove the manual copy |
| History migration misses data | The migration utility compares counts per table against Cosmos and refuses cutover on a mismatch |
| review-ui and core-pipeline drift apart on document shapes | Contract copy plus a test that fails on a difference |

## 12. What is still missing (2026-11-07)

In build order. Everything listed runs on Postgres today; nothing below is built.

1. **Storage interface (phase 2).** One `store` interface in core-pipeline, review-ui and db-utilities, with the Postgres code moved behind it. Without it nothing else can switch.
2. **Cosmos implementation (phases 2 and 6).** The containers in §4.1, `store/cosmos/` in each deployment, precomputed `metrics` docs for review-ui's cross-chart pages, and the rewrite of every SQL join (§4.4a). The largest item.
3. **`DB_BACKEND=both` writing to both stores** (phase 7), then the history migration utility with count checks, then cutover.
4. **Service Bus and separate workers (phase 4).** The API enqueues, workers pull one chart each (sessions, lock renewal, dead-letter), the `X-API-Key` check, and two images (API without models, worker with them).
5. **Outputs (phase 3).** Stage-to-stage data out of `staging/*.json` into the page docs; scratch wiped after each run; debug files to Blob as well as disk.
6. **Step-by-step APIs (phase 5).** `/v1/charts/{id}/ocr`, `/page-type`, `/member-verify`, `/dos`, `/v1/runs/{run_id}` with `?wait=`.
7. **Models and canon from Blob (phase 8).** `resources/manifest.json`, a publish script, fetch-at-start with fingerprints.
8. **Monitoring leftovers (7a).** LLM token wrapper, `/v1/metrics`, the review-ui Usage page.
9. **`infra/`.** Bicep for Cosmos containers, the queue and (later) AKS.
10. **AKS (phase 9).** Manifests or a Helm chart, KEDA scaling, Key Vault, ACR.